

# FLOWS - Flood Likelihood from pre-compiled Watershed Structure - Documentation

Thiago Ricardo Borges

September 7, 2026

# Contents

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                         | <b>3</b> |
| <b>2</b> | <b>Building</b>                                             | <b>4</b> |
| 2.1      | First thoughts . . . . .                                    | 4        |
| 2.1.1    | The challenge of flood prediction . . . . .                 | 4        |
| 2.1.2    | Why Flood Estimation for low-end hardware matters . . . . . | 4        |
| 2.1.3    | The Core idea . . . . .                                     | 5        |
| 2.1.4    | Proposed Approach . . . . .                                 | 5        |
| 2.1.5    | Long-term goal . . . . .                                    | 5        |
| 2.2      | Initial steps / handling geoTIF files . . . . .             | 5        |
| 2.3      | Finding basins . . . . .                                    | 11       |
| 2.4      | Analysing basins and handling infiltration . . . . .        | 37       |
| 2.5      | Applying results for the entire DEM . . . . .               | 58       |

# 1 Introduction

Introduction text

## 2 Building

Now we begin the actual process of making the model.

### 2.1 First thoughts

#### 2.1.1 The challenge of flood prediction

Flood prediction is one of the most computationally demanding problems in environmental modeling. Modern hydraulic models must simultaneously take in consideration terrain, rainfall, infiltration, surface runoff, river networks, soil properties, and the interaction between all these components through out time. Many systems rely on models that solve large systems of equations over millions of spatial cells. Depending on the resolution and simulation domain, these models may require powerful workstations, clusters and/or expensive computing infrastructure.

Examples include:

- The HEC-RAS hydraulic model, widely used for river and floodplain simulations.
- WRF-Hydro, which couples weather and hydraulic processes.
- National and regional flood forecasting systems operated by governments and research institutions

While these models achieve high levels of accuracy, they usually require significant resources, specialized knowledge and extensive input datasets

#### 2.1.2 Why Flood Estimation for low-end hardware matters

Many communities, schools, researchers, local governments and independent developers do not have access to high-performance computers. In emergency situations, decision-makers may benefit from a model that sacrifices some physical complexity in exchange for:

- Faster execution
- Lower hardware requirements
- Easier deployment
- Greater accessibility
- Large-scale applicability

The goal is not necessarily to replace advanced hydrodynamic simulation, but to provide a practical first-order estimate of flood-risk

### 2.1.3 The Core idea

Water naturally accumulates in topographic depressions. Instead of continuously solving fluid dynamics equations across an entire terrain, this project explores the possibility of pre-processing terrain into a network of natural storage basins. Each basin can be described by:

- Area
- Storage capacity
- Elevation - Volume relationship
- Overflow evaluation

Once this network is compiled, rainfall forecasts can be converted into water volumes and handled by our already calculated basin properties. This shifts much of the computation cost from runtime to a one-time preprocessing stage

### 2.1.4 Proposed Approach

- Obtain a Digital Elevation Model (DEM) with an acceptable resolution
- Identify topographic bowl-like depressions (here, we will just call them "basins")
- Calculate storage - volume relationships for each basin
- Build a graph representing basin connectivity
- Incorporate rainfall forecasts and infiltration estimates
- Estimate water accumulation and flood risk
- Test on real-life events.

### 2.1.5 Long-term goal

The objective is to investigate whether a graph-based representation of terrain storage can provide useful flood-risk estimates while remaining lightweight enough to run on consumer hardware. Such an approach could make flood-awareness tools more accessible to small municipalities, educational institutions, researchers and citizen initiatives.

## 2.2 Initial steps / handling geoTIF files

First of all, we got to check out the first item in our list: obtaining a DEM with acceptable resolution. Now this is where things can get tricky: some countries have decent, high-quality DEM's made with LiDAR and other fancy methods that will provide us with extremely accurate information. But for the big majority (including

me), those same resources are not easily accessible (if they can at all be found). In those cases, you would probably be best of with "open for public" elevation models such as Copernicus GLO-30 (that is very good, considering its freely available), but take in consideration that it may suffer from vegetation bias, and other inaccuracies that come with initiatives like this. Now, in my (the author) scenario, we are kind of lucky, since we have ANADEM.

ANADEM is a DEM brought to us by the Brazilian National Waters and Basic Sanitation Agency (ANA) in collaboration with the Federal University of Rio Grande do Sul (UFRGS). It is a refined version of Copernicus GLO-30, with reduced vegetation bias and developed with hydrological analysis in mind, available for South America.

Before continuing, it's important to remind the reader to be aware of DEM origins and quality, since those factors can and will affect the outcome of a model. It's also crucial to mention that, even though the goal of this documentation is to provide instructions and notes of FLOWS's construction to allow replication, the reader should have in mind that everything we use here was specifically made for Brazilian territory, so before adapting this project to your own needs please use parameters and resources for your own specific region/country.

Also let's have in mind that, for this project, hardware specs are so far:

- Lenovo Thinkpad E470
- Intel Core i5-7200U
- 8GB of DDR4 RAM
- 120GB SATA Solid State Drive
- Fedora Workstation 44

We need, as this project advances, to have this specs in mind.

Now, let's get to the really interesting stuff. So far, all that we want to do is find all the topographic depressions (let's call them basins), and get from them all the important data, like volume to water height relationship (probably the most important one so far). Later on, we will also analyze other factors like water infiltration and model accuracy.

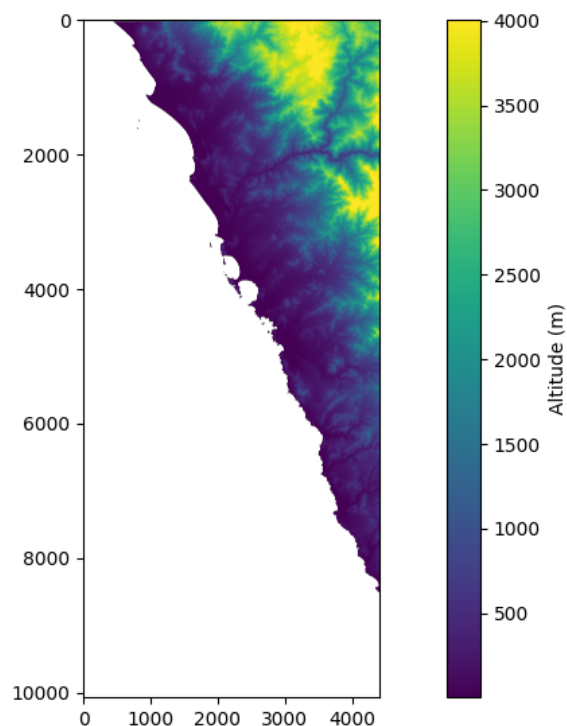
First things first, ANADEM is provided as several .tif tiles that, together, form the whole DEM, as a single big file could easily be 60GB in size or more (We are talking of South America here - I wasn't able to find a DEM focused on hydrological analysis for Brazilian territory alone) . So, to get started, we'll make a simple python script (we'll be working with this language for the rest of the guide since libraries for our specific use are widely available, thus making our whole journey easier) that will read a single tile and output it in a visual format:

```

1 import rasterio
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # This code tests reading a .tif file, and generates a cool
  image with it!
6
7 # We are here using a .tif file named mdt.tif.
8
9 with rasterio.open("mdt.tif") as src:
10 print("Bounds:", src.bounds)
11 print("CRS:", src.crs)
12 print("NoData:", src.nodata)
13 dem = src.read(1)
14
15 dem = np.where(dem == -9999, np.nan, dem)
16
17 valid = dem[~np.isnan(dem)]
18
19 print("Min:", np.min(valid))
20 print("Max:", np.max(valid))
21 print("Mean:", np.mean(valid))
22
23 plt.figure(figsize=(12,8))
24 plt.imshow(
25     dem,
26     vmin=np.percentile(valid, 2),
27     vmax=np.percentile(valid, 98)
28 )
29 plt.colorbar(label="Altitude (m)")
30 plt.show()
31

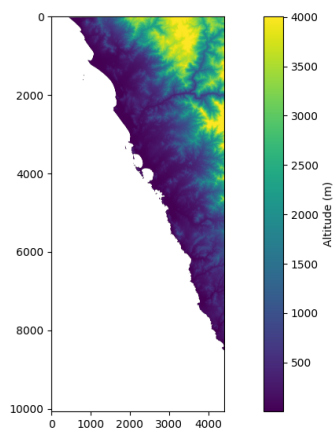
```

This code will generate us an output of this format:



(x, y) = (4.28e+03, 7.05e+03)  
[2.6e+02]

Now please take in consideration that this specific tile had some noData zones - areas of the tile with no specific information about them, so they appear in blank. This should not be important for us now, since this tile in specific is located on the west coast of South America, and a visual inspection gives us that this noData zones are actually the ocean. Therefore, we will not worry about them here in this part of the path.



(x, y) = (4.28e+03, 7.05e+03)  
[2.6e+02]





Now, we have to think how to actually take all of the info we've downloaded and turn it into an actually usable set of data. Some ideas would be:

- Load everything into a script and just find the basins → Well, not so fast there partner. As we've seen earlier, everything together will render us 60GB of information, all of that loaded at the same time, requiring a lot of storage and RAM. Since we don't have hardware like that available, we'll have to improvise.
- Load each individual tile one at a time → In theory this idea is pretty good! We would just need to load each downloaded tile individually and then find all the basins that are present in it. But there is a pretty important caveat: This solution is a victim to a widely discussed thing in hydrology and topography called Boundary Limitations → what would happen if a basin is cut in half by two adjacent tiles? This would generate inaccuracies that will be costly later. Therefore, not a totally viable solution.
- Load a central chunk of space and expand the borders if needed → How would this work: Basically we start to look for basins in an individual chunk in an individual tile that we will call the core box. We will try and look for all basins inside that area of space, and if any basin touches the boundaries delimited by the core box, we will expand our search space by adding a buffer to the sides. Then, we will look for basins that still belong to the original core box, but instead check whether any one touches the new boundaries delimited by the core box + buffer to the sides. If success, we get a file with a known area that does not suffer boundary limitations, but if we fail, we increase the buffer and try again. We repeat this process until we get a success.

This last idea seemed pretty good in theory and seems like the most straightforward option considering our scenario, but there are still some problems to it:

1. Terrain processing → Finding basins may seem straightforward, but actually applying all the things we learned into code, while having to mess with different file types and concepts that no one without a geography major would ever know (author included) is a completely different history.
2. How do we decide which basins should belong to the initial core box, and how do we deal with basins that belong (in area) to two different core boxes?
3. Way too big loops → What happens if a basin is just so big and we increase the buffer so much that the computer runs out of RAM?

But after some time brainstorming this topic, we are able to come up with a solution to each of these problems:

1. WhiteboxTools → Developed in Rust, WhiteboxTools is an advanced geospatial analysis library designed for hydrological processing. It comes with a range of tools, some made specifically for our objective of finding bowl-like depressions. Also, it can be used as a Python library! But not everything is perfect, and WhiteboxTools will still try to load everything we put at it on RAM without second thoughts. This will still break our computer if we try to load the entire DEM.

## What are the system requirements?

The answer to this question depends strongly on the type of analysis and data that you intend to process. However, generally we find performance to be optimal with a recommended minimum of 8-16GB of memory (RAM), a modern multi-core processor (e.g. 64-bit i5 or i7), and an solid-state-drive (SSD). It is likely that *WhiteboxTools* will have satisfactory performance on lower-spec systems if smaller datasets are being processed. Because *WhiteboxTools* reads entire raster datasets into system memory (for optimal performance, and in recognition that modern systems have increasingly larger amounts of fast RAM), this tends to be the limiting factor for the upper-end of data size successfully processed by the library. 64-bit operating systems are recommended and extensive testing

[https://jblindsay.github.io/wbt\\_book/print.html](https://jblindsay.github.io/wbt_book/print.html)

Page 633 of 667

2. Geometry! → Every geometric shape has a unique point called the centroid (in some cases also known as a center of mass). What we can do is check, for each basin with a centroid belonging inside the core box boundaries, whether it touches the borders of the bigger core box + buffer boundaries. And since the centroid is unique, we can guarantee that each basin will belong in a single core box.
3. A little bit of math (and hope) → if we sum all the tile sizes, we get that our whole DEM, for the entire South America, weights around 63GB, Following that logic, the biggest floodable plain also in South America (Pantanal) would weight around 0.9GB in size if it were its own big basin. So in theory, and big emphasis in theory, we should be able to run this program if we slice the big DEM in smaller individual core boxes the right way and optimize resource usage on our computer. But there is only one way to find out!

But you may still be wondering to yourself: "Okay, but how will we make our code look at all of our downloaded tiles and treat it as one big individual DEM without exceeding RAM and storage?" - for that question, we will also introduce the concept of a VRT (Virtual Raster).

A VRT is a lightweight XML-based file created by the GDAL ecosystem that acts essentially as a virtual mosaic. Instead of holding raw pixel data, it stores metadata and instructions on how multiple individual downloaded tiles should be aligned and

organized to appear as a single, continuous, massive image. It basically works as follows (in very simple terms):

- Zero Space Duplication: We don't need to physically merge all tiles into a 60GB monster on disk. The VRT dynamically points to where each original tile is saved.
- On-Demand Reading: When the code starts processing the topography, it reads through the VRT, managing block reading and streaming only the necessary pixels directly from the disk at that exact moment.
- Seamless Global View: To the algorithm, the VRT makes the entire dataset look like a single continuous file, bypassing memory bottlenecks and removing the need to manage tile transitions manually.

So with all of that in mind, let's get started.

## 2.3 Finding basins

First of all, we have to create our VRT. There are several methods out there, but here it was chosen to use the terminal tool. To install it, you must follow instructions for your specific system. In this guide, we are on Fedora Workstation 44:

```
sudo dnf update && sudo dnf install gdal
```

After that, we must go to the folder containing all of our downloaded .tif files, and run the following command:

```
gdalbuildvrt <output file>.vrt *.tif
```

In this guide, we've chosen to name the output file as simply output.vrt, therefore we ran:

```
gdalbuildvrt output.vrt *.tif
```

```

untitledmacaw@fedora:~/Downloads$ ls
anadem_mgrs.zip  anadem_v1_18N.tif  anadem_v1_20F.tif  anadem_v1_21K.tif  anadem_v1_23J.tif
anadem_tiles.png  anadem_v1_18P.tif  anadem_v1_20G.tif  anadem_v1_21L.tif  anadem_v1_23K.tif
anadem_v1_17L.tif  anadem_v1_19F.tif  anadem_v1_20H.tif  anadem_v1_21M.tif  anadem_v1_23L.tif
anadem_v1_17M.tif  anadem_v1_19G.tif  anadem_v1_20J.tif  anadem_v1_21N.tif  anadem_v1_23M.tif
anadem_v1_17N.tif  anadem_v1_19H.tif  anadem_v1_20K.tif  anadem_v1_21P.tif  anadem_v1_24K.tif
anadem_v1_18F.tif  anadem_v1_19J.tif  anadem_v1_20M.tif  anadem_v1_21Q.tif  anadem_v1_24L.tif
anadem_v1_18G.tif  anadem_v1_19K.tif  anadem_v1_20N.tif  anadem_v1_21R.tif  anadem_v1_24M.tif
anadem_v1_18H.tif  anadem_v1_19L.tif  anadem_v1_20P.tif  anadem_v1_21S.tif  anadem_v1_25L.tif
anadem_v1_18I.tif  anadem_v1_19M.tif  anadem_v1_20Q.tif  anadem_v1_21T.tif  anadem_v1_25M.tif
anadem_v1_18J.tif  anadem_v1_19N.tif  anadem_v1_20R.tif  anadem_v1_21U.tif  RJ_2016.pdf
anadem_v1_18K.tif  anadem_v1_19P.tif  anadem_v1_20S.tif  anadem_v1_21V.tif  Sistema-Brasileiro-de-Classificacao-de-Solos-2025.pdf
untitledmacaw@fedora:~/Downloads$ gdalbuildvrt output.vrt *.tif
0...10...20...30...40...50...60...70...80...90...100 - done.
untitledmacaw@fedora:~/Downloads$ ls
anadem_mgrs.zip  anadem_v1_18P.tif  anadem_v1_20J.tif  anadem_v1_21N.tif  anadem_v1_24K.tif
anadem_tiles.png  anadem_v1_19F.tif  anadem_v1_20K.tif  anadem_v1_21P.tif  anadem_v1_24L.tif
anadem_v1_17L.tif  anadem_v1_19G.tif  anadem_v1_20L.tif  anadem_v1_21Q.tif  anadem_v1_24M.tif
anadem_v1_17M.tif  anadem_v1_19H.tif  anadem_v1_20M.tif  anadem_v1_21R.tif  anadem_v1_25L.tif
anadem_v1_17N.tif  anadem_v1_19J.tif  anadem_v1_20N.tif  anadem_v1_21S.tif  anadem_v1_25M.tif
anadem_v1_18F.tif  anadem_v1_19K.tif  anadem_v1_20P.tif  anadem_v1_21T.tif  output.vrt
anadem_v1_18G.tif  anadem_v1_19L.tif  anadem_v1_20Q.tif  anadem_v1_21U.tif  RJ_2016.pdf
anadem_v1_18H.tif  anadem_v1_19M.tif  anadem_v1_20R.tif  anadem_v1_21V.tif  Sistema-Brasileiro-de-Classificacao-de-Solos-2025.pdf
anadem_v1_18I.tif  anadem_v1_19N.tif  anadem_v1_20S.tif  anadem_v1_21W.tif  anadem_v1_23K.tif
anadem_v1_18J.tif  anadem_v1_19P.tif  anadem_v1_20T.tif  anadem_v1_21X.tif  anadem_v1_23L.tif
anadem_v1_18K.tif  anadem_v1_19Q.tif  anadem_v1_20U.tif  anadem_v1_21Y.tif  anadem_v1_23M.tif
untitledmacaw@fedora:~/Downloads$

```

The next thing we should do is also install Whitebox:

```
pip install whitebox
```

```

(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ pip install whitebox
Collecting whitebox
  Downloading whitebox-2.3.6-py2.py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: Click>=6.0 in ./venv/lib64/python3.14/site-packages (from whitebox) (8.4.1)
Downloading whitebox-2.3.6-py2.py3-none-any.whl (74 kB)
Installing collected packages: whitebox
Successfully installed whitebox-2.3.6

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ pip freeze > requirements.txt
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$

```

Perfect! So now let's move on to the next step. Our current code workflow can be described as follows:

1. Chose 4 values (min x, min y, max x, max y) that will delimitate our core box
2. Run a loop where we:
  - Increase the buffer by a fixed value
  - Convert the piece from the original VRT delimited by our core box + buffer from referencing system EPSG:4326 (degrees) to referencing system EPSG:5800 (metres). This is important because calculations later on involving degrees will gives us inaccuracies.
  - Check if the current buffer is way too big (we will set a buffer limit for that)
  - Find all bowl-like depressions + their contribution areas and, for each one, find its centroid
  - Filter results where the centroid belongs to the original core box

- For those results, we check if any one touches the border of the core box + buffer area
  - If yes, we run another cycle. If not, we break the loop
3. We save our results and analyse each individual basin.

To process the topography and find bowl-like depressions, we will rely on a sequence of functions provided by WhiteboxToos:

- **breach\_single\_cell\_pits**: Resolves single-cell topographic pits in the raw DEM via selective breaching rather than filling. This ensures continuous flow routing while preventing the over-smoothing of surrounding terrain.
- **d8\_pointer**: Calculates flow directions using the standard D8 algorithm, assigning each grid cell an integer pointer toward its steepest downslope neighbor.
- **sink**: Identifies local depressions and pour points across the DEM where surface runoff accumulates and cannot drain further.
- **watershed**: Delineates the full contributing catchment area (basin) for each pour point identified by **sink** using the D8 pointer grid.
- **depth\_in\_sink**: Computes the maximum storage depth for each basin grid cell by measuring the elevation difference between the cell and its basin's spill elevation.

After catchment boundaries are calculated, we use NumPy array masking to isolate basins assigned to the current core box and strip away unwanted terrain without messing with basin geometry:

1. **Centroid Classification**: Using `scipy.ndimage.center_of_mass`, we calculate the spatial centroid  $(x, y)$  of each basin raster in pixel coordinates and project them to ground coordinates in meters (EPSG:5880). If a centroid falls strictly within the core tile bounds ( $c_{\min x} \leq x \leq c_{\max x}$  and  $c_{\min y} \leq y \leq c_{\max y}$ ), its basin ID is stored in the set `owned_by_core_ids`.
2. **Boolean Mask Generation (`np.isin`)**: We convert `owned_by_core_ids` to a 1D NumPy array (`core_ids_array`) and evaluate `np.isin(basins_data, core_ids_array)`. This generates a 2D boolean grid (`mask`) where pixels belonging to target basins are set to `True`, and adjacent basins leaking into the buffer are set to `False`.
3. **Raster Filtering (`np.where`)**: Passing the boolean mask to `np.where(mask, basins_data, nodata_val)` replaces all `False` pixels with `nodata_val` (or 0). The identical mask is applied to the depth output (`filtered_depth = np.where(mask, depth_data, nodata_val_depth)`). This cleanly clips away all non-core basins while maintaining the intact vector geometry of every basin originating inside the core tile.

With that in mind, let's start translating that to code. We are able to initially come up with this:

```
1 import os
2 import geopandas as gpd
3 from shapely.geometry import box
4 from osgeo import gdal
5 from whitebox import WhiteboxTools
6 import rasterio
7 import numpy as np
8 import scipy.ndimage as ndimage
9 import matplotlib.pyplot as plt
10 import matplotlib.patches as patches
11 from matplotlib.colors import ListedColormap
12
13 gdal.UseExceptions()
14
15 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
16     """
17     This will cut the DEM according to topography, preserving
18     basin boundaries (AKA no basins get cut)
19     Criteria for cutting: All basins with a centroid that
20     belongs to core tile A will be grouped together on the same
21     TIF
22
23     :param path_vrt: Path to our .vrt file
24     :param out_dir: Path for our output
25     :param core_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
26     :param tile_id: Unique tile ID
27
28     """
29
30     os.makedirs(out_dir, exist_ok=True)
31     minx, miny, maxx, maxy = core_bounds_deg
32
33     # Projects Core Tile limit from 4326 (degrees) to 5800 (
34     # meters)
35     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
36 maxx, maxy)]}, crs="EPSG:4326")
37     df_core_5800 = df_core.to_crs("EPSG:5800")
38     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
39 maxx, maxy) em metros
40
41     # Dynamic buffer initial setup
```

```

36     buffer_current = 0.1
37     buffer_steps = 0.1
38     buffer_limit = 4
39     sucess = False
40
41     wbt = WhiteboxTools()
42     wbt.set_working_dir(os.path.abspath(out_dir))
43     wbt.verbose = False
44
45     while not sucess and buffer_current <= buffer_limit:
46         print(f"[{tile_id}] Trying to process with a margin of {
buffer_current}")
47
48         # Define limits
49         buf_minx = minx - buffer_current
50         buf_miny = miny - buffer_current
51         buf_maxx = maxx + buffer_current
52         buf_maxy = maxy + buffer_current
53
54         # Names of temporary files
55         filename_dem = f"temp_{tile_id}_dem.tif"
56         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
57         filename_d8 = f"temp_{tile_id}_d8.tif"
58         filename_sinks = f"temp_{tile_id}_sinks.tif"
59         filename_basins = f"basins_final_{tile_id}.tif"
60         filename_depth = f"depth_final_{tile_id}.tif"
61
62         path_dem = os.path.join(out_dir, filename_dem)
63         path_basins = os.path.join(out_dir, filename_basins)
64
65         try:
66             # Cuts OG VRT
67             gdal.Warp(
68                 destNameOrDestDS=path_dem,
69                 srcDSOrSrcDSTab=path_vrt,
70                 format='GTiff',
71                 outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
72                 outputBoundsSRS='EPSG:4326',
73                 dstSRS='EPSG:5880',
74                 xRes=30, yRes=30,
75                 creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]
76             )

```

```

77
78     # WhiteboxTools pipeline here
79     print(f"        -> Running WhiteboxTools ...")
80     wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)
81     wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
82     wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
83     wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
84
85     # Colision Test pipeline here
86
87     print(f"        -> Running colision test ...")
88
89     with rasterio.open(path_basins) as src:
90         basins_data = src.read(1)
91         transform = src.transform
92
93     # a) Grabs IDS from all basins that touch the bordes
of the current matrix (core box + current buffer)
94     top_edge = basins_data[0, :]
95     bottom_edge = basins_data[-1, :]
96     left_edge = basins_data[:, 0]
97     right_edge = basins_data[:, -1]
98
99     edge_ids = set(np.unique(np.concatenate([top_edge,
bottom_edge, left_edge, right_edge])))
100     edge_ids.discard(0) # Removes noData
101
102     # b) Calculates the centroid of each basin present
inside the raster file
103     unique_ids = np.unique(basins_data)
104     unique_ids = unique_ids[unique_ids > 0] # Removes
noData (I hope)
105
106     centers = ndimage.center_of_mass(basins_data > 0,
basins_data, unique_ids)
107
108     owned_by_core_ids = set()
109     c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
110
111     # c) Filters which basins have their centroid INSIDE

```



```

    core tile
112         for basin_id, (row_center, col_center) in zip(
unique_ids, centers):
113             # Converts pixel (line, column) to geo cords (x,
y in meters)
114             x_coord, y_coord = transform * (col_center,
row_center)
115
116             if (c_minx <= x_coord <= c_maxx) and (c_miny <=
y_coord <= c_maxy):
117                 owned_by_core_ids.add(basin_id)
118
119             # d) Is there any basin with centroid inside core
tile that leaks outside of the buffer?
120             leaking_basins = owned_by_core_ids.intersection(
edge_ids)
121
122             if len(leaking_basins) > 0:
123                 print(f"[FAIL] {len(leaking_basins)} basin(s)
leaked outside current buffer")
124                 buffer_current += buffer_steps
125
126             else:
127                 print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
128
129                 wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
130
131                 sucess = True
132
133
134
135             except Exception as e:
136                 print(f"[ERROR]: {e}")
137                 print(f"    -> Ending runtime ...")
138                 break
139
140             # File cleanup
141             temp_files = [filename_dem, filename_dem_clean,
filename_d8, filename_sinks]
142             if not sucess:
143                 temp_files.extend([filename_basins, filename_depth])
# Deletes if failed

```

```

144
145     for f in temp_files:
146         path_f = os.path.join(out_dir, f)
147         if os.path.exists(path_f):
148             os.remove(path_f)
149
150     if not sucess and buffer_current >= buffer_limit:
151         print(f"[WARNING]: Buffer limit reached. Consider
152 revising terrain.")
153
154 if True:
155     PATH_VRT = '~/Downloads/output.vrt'
156     OUT_DIR = 'test_basins_dynamic'
157     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
158
159     dynamic_cut(
160         path_vrt=PATH_VRT,
161         out_dir=OUT_DIR,
162         core_bounds_deg=BOUNDS_TESTING,
163         tile_id="test_01"
164     )
165

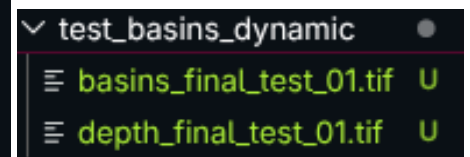
```

Here, we chose a small box just to run a quick check (we will keep using this same area until we're okay to start handling our entire dataset). Our output looked something like this:

```

[test_01] Trying to process with a margin of 0.1
-> Running WhiteboxTools ...
-> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
-> Running WhiteboxTools ...
-> Running colision test ...
[SUCESS] All interesting basins fit within a buffer of 0.2

```



But how can we actually verify that our code is indeed working? Wouldn't it be cool if we could somehow get a visual output of basins and out core box and core box + buffer at the end of runtime?

For that to work, we have to adapt our code a little bit:

```

1 import os
2 import geopandas as gpd
3 from shapely.geometry import box
4 from osgeo import gdal
5 from whitebox import WhiteboxTools

```

```

6 import rasterio
7 import numpy as np
8 import scipy.ndimage as ndimage
9 import matplotlib.pyplot as plt
10 import matplotlib.patches as patches
11 from matplotlib.colors import ListedColormap
12
13 gdal.UseExceptions()
14
15 def plot_tile_diagnostics(
16     basins_data,
17     transform,
18     core_bounds_5800,
19     buf_deg_bounds,
20     centers,
21     unique_ids,
22     owned_by_core_ids,
23     tile_id,
24 ):
25     """
26     Takes our results from the dynamic cutting and outputs them
27     as a visual .png file so we can have visual
28     proof that all centroids are contained inside the core box
29     It is advised to read the dynamic_cut function before
30     reading this one (down below).
31
32     :param basins_data: Our basins raster array
33     :param transform: Our transformation matrix (provided by
34     rasterio)
35     :param core_bounds_5800: Tuple (minx, miny, maxx, maxy) that
36     defines our core box limits
37     :param buf_deg_bounds: Tuple (buf_minx, buf_miny, buf_maxx,
38     buf_maxy) that defines our core box + buffer limits
39     :param centers: List with all coordinates (line and column)
40     from each of the basins centroids
41     :param unique_ids: Array containing all of the unique basin
42     IDs present in the image (removing 0)
43     :param tile_id: Our tile ID
44
45     """
46
47     # WARNING: DO NOT REMOVE FIG.
48     fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
49

```

```

43 # 1. Generating unique colors for each ID
44 # Generates random colors for each ID to try to avoid visual
   problems with neighbor basins
45 max_id = int(basins_data.max())
46 np.random.seed(
47     42
48 )
49 color_array = np.random.rand(max_id + 1, 4)
50 color_array[
51     0
52 ] = [ # Defines background color (ID 0 = noData)
53     0,
54     0,
55     0,
56     0,
57 ]
58 custom_cmap = ListedColormap(color_array)
59
60 # 2. Plots basins raster with custom colors
61 im = ax.imshow(
62     basins_data,
63     cmap=custom_cmap,
64     interpolation='nearest', # Avoids blurring
65     extent=[
66         transform[2],
67         transform[2] + transform[0] * basins_data.shape[1],
68         transform[5] + transform[4] * basins_data.shape[0],
69         transform[5],
70     ],
71 )
72
73 buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
74 df_buf = gpd.GeoDataFrame(
75     {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy)
76 ]}],
77     crs='EPSG:4326',
78 )
79 b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
total_bounds
80
81 c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
82
83 # Draws Core Box limits
84 core_rect = patches.Rectangle(
85     (c_minx, c_miny),

```

```

84         c_maxx - c_minx,
85         c_maxy - c_miny,
86         linewidth=2.5,
87         edgecolor='red',
88         facecolor='none',
89         linestyle='--',
90         label='Core Tile',
91     )
92     ax.add_patch(core_rect)
93
94     # Draws Core Box + Buffer tile
95     buf_rect = patches.Rectangle(
96         (b_minx, b_miny),
97         b_maxx - b_minx,
98         b_maxy - b_miny,
99         linewidth=2,
100        edgecolor='blue',
101        facecolor='none',
102        label='Buffer Tile (Final)',
103    )
104    ax.add_patch(buf_rect)
105
106    # Plots centroids
107    first_core = True
108    for basin_id, (row_center, col_center) in zip(unique_ids,
109    centers):
110
111        x_coord, y_coord = transform * (col_center, row_center)
112
113        if basin_id in owned_by_core_ids:
114            label_text = 'Centroid Core' if first_core else ''
115            ax.plot(
116                x_coord,
117                y_coord,
118                marker='*',
119                color='black',
120                markersize=5,
121                markeredgecolor='white',
122                markeredgewidth=0.5,
123                label=label_text,
124            )
125            first_core = False
126
127        else:
128            ax.plot(

```

```

127         x_coord,
128         y_coord,
129         marker='o',
130         color='dimgray',
131         markersize=2,
132         alpha=0.4,
133     )
134
135     # Adjust zoom
136     ax.set_xlim(b_minx, b_maxx)
137     ax.set_ylim(b_miny, b_maxy)
138
139     # Graphic aesthetics
140     ax.set_facecolor('#f4f4f4')
141     ax.grid(True, linestyle=':', alpha=0.5, color='gray')
142     plt.title(
143         f'Basins check - Tile: {tile_id}',
144         fontsize=14,
145         fontweight='bold',
146     )
147     plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
148     plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
149     plt.legend(loc='upper right', frameon=True, facecolor='white', shadow=True)
150
151     plt.tight_layout()
152     plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
153     plt.close()
154     print(
155         f'        -> Image stored as: diagnostic_{tile_id}.png'
156     )
157
158 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
159     """
160     This will cut the DEM according to topography, preserving
161     basin boundaries (AKA no basins get cut)
162     Criteria for cutting: All basins with a centroid that
163     belongs to core tile A will be grouped together on the same
164     TIF
165
166     :param path_vrt: Path to our .vrt file
167     :param out_dir: Path for our output
168     :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
169     :param tile_id: Unique tile ID

```

```

167
168     """
169
170     os.makedirs(out_dir, exist_ok=True)
171     minx, miny, maxx, maxy = core_bounds_deg
172
173     # Projects Core Tile limist from 4326 (degress) to 5800 (
174     # meters)
175     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
176     maxx, maxy)]}, crs="EPSG:4326")
177     df_core_5800 = df_core.to_crs("EPSG:5880")
178     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
179     maxx, maxy) em metros
180
181     # Dynamic buffer initial setup
182     buffer_current = 0.1
183     buffer_steps = 0.1
184     buffer_limit = 4
185     sucess = False
186
187     wbt = WhiteboxTools()
188     wbt.set_working_dir(os.path.abspath(out_dir))
189     wbt.verbose = False
190
191     while not sucess and buffer_current <= buffer_limit:
192         print(f"[{tile_id}] Trying to process with a margin of {
193         buffer_current}")
194
195         # Define limits
196         buf_minx = minx - buffer_current
197         buf_miny = miny - buffer_current
198         buf_maxx = maxx + buffer_current
199         buf_maxy = maxy + buffer_current
200
201         # Names of temporary files
202         filename_dem = f"temp_{tile_id}_dem.tif"
203         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
204         filename_d8 = f"temp_{tile_id}_d8.tif"
205         filename_sinks = f"temp_{tile_id}_sinks.tif"
206         filename_basins = f"basins_final_{tile_id}.tif"
207         filename_depth = f"depth_final_{tile_id}.tif"
208
209         path_dem = os.path.join(out_dir, filename_dem)
210         path_basins = os.path.join(out_dir, filename_basins)

```

```

207
208     try:
209         # Cuts OG VRT
210         gdal.Warp(
211             destNameOrDestDS=path_dem,
212             srcDSOrSrcDSTab=path_vrt,
213             format='GTiff',
214             outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
215             outputBoundsSRS='EPSG:4326',
216             dstSRS='EPSG:5880',
217             xRes=30, yRes=30,
218             creationOptions=['COMPRESS=DEFLATE', 'TILED=YES'
]
219         )
220
221         # WhiteboxTools pipeline here
222         print(f"        -> Running WhiteboxTools ...")
223         wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)
224         wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
225         wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
226         wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
227
228         # Colision Test pipeline here
229
230         print(f"        -> Running colision test ...")
231
232         with rasterio.open(path_basins) as src:
233             basins_data = src.read(1)
234             transform = src.transform
235
236         # a) Grabs IDS from all basins that touch the bordes
of the current matrix (core box + current buffer)
237         top_edge = basins_data[0, :]
238         bottom_edge = basins_data[-1, :]
239         left_edge = basins_data[:, 0]
240         right_edge = basins_data[:, -1]
241
242         edge_ids = set(np.unique(np.concatenate([top_edge,
bottom_edge, left_edge, right_edge])))

```



```

243         edge_ids.discard(0) # Removes noData
244
245         # b) Calculates the centroid of each basin present
inside the raster file
246         unique_ids = np.unique(basins_data)
247         unique_ids = unique_ids[unique_ids > 0] # Removes
noData (I hope)
248
249         centers = ndimage.center_of_mass(basins_data > 0,
basins_data, unique_ids)
250
251         owned_by_core_ids = set()
252         c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
253
254         # c) Filters which basins have their centroid INSIDE
core tile
255         for basin_id, (row_center, col_center) in zip(
unique_ids, centers):
256             # Converts pixel (line, column) to geo cords (x,
y in meters)
257             x_coord, y_coord = transform * (col_center,
row_center)
258
259             if (c_minx <= x_coord <= c_maxx) and (c_miny <=
y_coord <= c_maxy):
260                 owned_by_core_ids.add(basin_id)
261
262         # d) Is there any basin with centroid inside core
tile that leaks outside of the buffer?
263         leaking_basins = owned_by_core_ids.intersection(
edge_ids)
264
265         if len(leaking_basins) > 0:
266             print(f"[FAIL] {len(leaking_basins)} basin(s)
leaked outside current buffer")
267             buffer_current += buffer_steps
268
269         else:
270             print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
271
272         wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
273

```

```

274         # testing quick visuals
275         print(f"        -> Generating image ...")
276         buf_bounds = (buf_minx, buf_miny, buf_maxx,
buf_maxy)
277         plot_tile_diagnostics(
278             basins_data,
279             transform,
280             core_bounds_5800,
281             buf_bounds,
282             centers,
283             unique_ids,
284             owned_by_core_ids,
285             tile_id,
286         )
287         sucess = True
288
289
290
291     except Exception as e:
292         print(f"[ERROR]: {e}")
293         print(f"        -> Ending runtime ...")
294         break
295
296     #
297     temp_files = [filename_dem, filename_dem_clean,
filename_d8, filename_sinks]
298     if not sucess:
299         temp_files.extend([filename_basins, filename_depth])
300     # Deletes if failed
301
302     for f in temp_files:
303         path_f = os.path.join(out_dir, f)
304         if os.path.exists(path_f):
305             os.remove(path_f)
306
307     if not sucess and buffer_current >= buffer_limit:
308         print(f"[WARNING]: Buffer limit reached. Consider
revising terrain.")
309
310 if True:
311     PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
312     OUT_DIR = 'test_basins_dynamic'
313     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)

```

```

314     dynamic_cut(
315         path_vrt=PATH_VRT,
316         out_dir=OUT_DIR,
317         core_bounds_deg=BOUNDS_TESTING,
318         tile_id="test_01"
319     )
320
321

```

What this code does is basically, at the end of the loop, if we get a success, we call a function to generate a visual graph of our results, and output it as a .png file.

```

[test_01] Trying to process with a margin of 0.1
-> Running WhiteboxTools ...
-> Running collision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
-> Running WhiteboxTools ...
-> Running collision test ...
[SUCCESS] All interesting basins fit within a buffer of 0.2
-> Generating image ...
-> Image stored as: diagnostic_test_01.png

```

But before we check result yet, we have to exclude the basins that are outside our area of interest.

To remove those other basins, we adapt the code some more:

```

1  import os
2  import geopandas as gpd
3  from shapely.geometry import box
4  from osgeo import gdal
5  from whitebox import WhiteboxTools
6  import rasterio
7  import numpy as np
8  import scipy.ndimage as ndimage
9  import matplotlib.pyplot as plt
10 import matplotlib.patches as patches
11 from matplotlib.colors import ListedColormap
12
13 gdal.UseExceptions()
14
15 def plot_tile_diagnostics(
16     basins_data,
17     transform,
18     core_bounds_5800,
19     buf_deg_bounds,

```

```

20     centers,
21     unique_ids,
22     owned_by_core_ids,
23     tile_id,
24 ):
25     """
26     Takes our results from the dynamic cutting and outputs them
27     as a visual .png file so we can have visual
28     proof that all centroids are contained inside the core box
29     It is advised to read the dynamic_cut function before
30     reading this one (down below).
31
32     :param basins_data: Our basins raster array
33     :param transform: Our transformation matrix (provided by
34     rasterio)
35     :param core_bounds_5800: Tuple (minx, miny, maxx, maxy) that
36     defines our core box limits
37     :param buf_deg_bounds: Tuple (buf_minx, buf_miny, buf_maxx,
38     buf_maxy) that defines our core box + buffer limits
39     :param centers: List with all coordinates (line and column)
40     from each of the basins centroids
41     :param unique_ids: Array containing all of the unique basin
42     IDs present in the image (removing 0)
43     :param tile_id: Our tile ID
44
45     """
46
47     # WARNING: DO NOT REMOVE FIG.
48     fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
49
50     # 1. Generating unique colors for each ID
51     # Generates random colors for each ID to try to avoid visual
52     # problems with neighbor basins
53     max_id = int(basins_data.max())
54     np.random.seed(
55         42
56     )
57     color_array = np.random.rand(max_id + 1, 4)
58     color_array[
59         0
60     ] = [ # Defines background color (ID 0 = noData)
61         0,
62         0,
63         0,
64         0,
65     ]

```

```

56         0,
57     ]
58     custom_cmap = ListedColormap(color_array)
59
60     # 2. Plots basins raster with custom colors
61     im = ax.imshow(
62         basins_data,
63         cmap=custom_cmap,
64         interpolation='nearest', # Avoids blurring
65         extent=[
66             transform[2],
67             transform[2] + transform[0] * basins_data.shape[1],
68             transform[5] + transform[4] * basins_data.shape[0],
69             transform[5],
70         ],
71     )
72
73     buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
74     df_buf = gpd.GeoDataFrame(
75         {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy)
76         ]}],
77         crs='EPSG:4326',
78     )
79     b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
total_bounds
80
81     # Draws Core Box limits
82     core_rect = patches.Rectangle(
83         (c_minx, c_miny),
84         c_maxx - c_minx,
85         c_maxy - c_miny,
86         linewidth=2.5,
87         edgecolor='red',
88         facecolor='none',
89         linestyle='--',
90         label='Core Tile',
91     )
92     ax.add_patch(core_rect)
93
94     # Draws Core Box + Buffer tile
95     buf_rect = patches.Rectangle(
96         (b_minx, b_miny),
97         b_maxx - b_minx,

```

```

108         b_maxy - b_miny,
109         linewidth=2,
110         edgecolor='blue',
111         facecolor='none',
112         label='Buffer Tile (Final)',
113     )
114     ax.add_patch(buf_rect)
115
116     # Plots centroids
117     first_core = True
118     for basin_id, (row_center, col_center) in zip(unique_ids,
119 centers):
120         x_coord, y_coord = transform * (col_center, row_center)
121
122         if basin_id in owned_by_core_ids:
123             label_text = 'Centroid Core' if first_core else ''
124             ax.plot(
125                 x_coord,
126                 y_coord,
127                 marker='',
128                 color='black',
129                 markersize=5,
130                 markeredgecolor='white',
131                 markeredgewidth=0.5,
132                 label=label_text,
133             )
134             first_core = False
135
136         else:
137             ax.plot(
138                 x_coord,
139                 y_coord,
140                 marker='',
141                 color='dimgray',
142                 markersize=2,
143                 alpha=0.4,
144             )
145
146     # Ajust zoom
147     ax.set_xlim(b_minx, b_maxx)
148     ax.set_ylim(b_miny, b_maxy)
149
150     # Graphic aesthetics
151     ax.set_facecolor('#f4f4f4')

```

```

141     ax.grid(True, linestyle=':', alpha=0.5, color='gray')
142     plt.title(
143         f'Basins check - Tile: {tile_id}',
144         fontsize=14,
145         fontweight='bold',
146     )
147     plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
148     plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
149     plt.legend(loc='upper right', frameon=True, facecolor='white',
150               shadow=True)
151
152     plt.tight_layout()
153     plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
154     plt.close()
155     print(
156         f'        -> Image stored as: diagnostic_{tile_id}.png'
157     )
158
159 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
160     """
161     This will cut the DEM according to topography, preserving
162     basin boundaries (AKA no basins get cut)
163     Criteria for cutting: All basins with a centroid that
164     belongs to core tile A will be grouped together on the same
165     TIF
166
167     :param path_vrt: Path to our .vrt file
168     :param out_dir: Path for our output
169     :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
170     :param tile_id: Unique tile ID
171
172     """
173
174     os.makedirs(out_dir, exist_ok=True)
175     minx, miny, maxx, maxy = core_bounds_deg
176
177     # Projects Core Tile limist from 4326 (degress) to 5800 (
178     # meters)
179     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
180         maxx, maxy)]}, crs="EPSG:4326")
181     df_core_5800 = df_core.to_crs("EPSG:5880")
182     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
183     maxx, maxy) em metros

```

```

178     # Dynamic buffer initial setup
179     buffer_current = 0.1
180     buffer_steps = 0.1
181     buffer_limit = 4
182     sucess = False
183
184     wbt = WhiteboxTools()
185     wbt.set_working_dir(os.path.abspath(out_dir))
186     wbt.verbose = False
187
188     while not sucess and buffer_current <= buffer_limit:
189         print(f"[{tile_id}] Trying to process with a margin of {
buffer_current}")
190
191         # Define limits
192         buf_minx = minx - buffer_current
193         buf_miny = miny - buffer_current
194         buf_maxx = maxx + buffer_current
195         buf_maxy = maxy + buffer_current
196
197         # Names of temporary files
198         filename_dem = f"temp_{tile_id}_dem.tif"
199         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
200         filename_d8 = f"temp_{tile_id}_d8.tif"
201         filename_sinks = f"temp_{tile_id}_sinks.tif"
202         filename_basins = f"basins_final_{tile_id}.tif"
203         filename_depth = f"depth_final_{tile_id}.tif"
204
205         path_dem = os.path.join(out_dir, filename_dem)
206         path_basins = os.path.join(out_dir, filename_basins)
207
208         try:
209             # Cuts OG VRT
210             gdal.Warp(
211                 destNameOrDestDS=path_dem,
212                 srcDSOrSrcDSTab=path_vrt,
213                 format='GTiff',
214                 outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
215                 outputBoundsSRS='EPSG:4326',
216                 dstSRS='EPSG:5880',
217                 xRes=30, yRes=30,
218                 creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]

```



```

219         )
220
221         # WhiteboxTools pipeline here
222         print(f"        -> Running WhiteboxTools ...")
223         wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)
224         wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
225         wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
226         wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
227
228         # Collision Test pipeline here
229
230         print(f"        -> Running collision test ...")
231
232         with rasterio.open(path_basins) as src:
233             basins_data = src.read(1)
234             transform = src.transform
235
236         # a) Grabs IDS from all basins that touch the bordes
of the current matrix (core box + current buffer)
237         top_edge = basins_data[0, :]
238         bottom_edge = basins_data[-1, :]
239         left_edge = basins_data[:, 0]
240         right_edge = basins_data[:, -1]
241
242         edge_ids = set(np.unique(np.concatenate([top_edge,
bottom_edge, left_edge, right_edge])))
243         edge_ids.discard(0) # Removes noData
244
245         # b) Calculates the centroid of each basin present
inside the raster file
246         unique_ids = np.unique(basins_data)
247         unique_ids = unique_ids[unique_ids > 0] # Removes
noData (I hope)
248
249         centers = ndimage.center_of_mass(basins_data > 0,
basins_data, unique_ids)
250
251         owned_by_core_ids = set()
252         c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
253

```

```

254         # c) Filters which basins have their centroid INSIDE
        core tile
255         for basin_id, (row_center, col_center) in zip(
unique_ids, centers):
256             # Converts pixel (line, column) to geo cords (x,
y in meters)
257             x_coord, y_coord = transform * (col_center,
row_center)
258
259             if (c_minx <= x_coord <= c_maxx) and (c_miny <=
y_coord <= c_maxy):
260                 owned_by_core_ids.add(basin_id)
261
262         # d) Is there any basin with centroid inside core
tile that leaks outside of the buffer?
263         leaking_basins = owned_by_core_ids.intersection(
edge_ids)
264
265         if len(leaking_basins) > 0:
266             print(f"[FAIL] {len(leaking_basins)} basin(s)
leaked outside current buffer")
267             buffer_current += buffer_steps
268
269         else:
270             print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
271
272         wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
273
274         # Removing excess basins
275         print(f"        -> Applying mask to keep only basins
of interest ...")
276
277         # a) Transforms the valid IDs set into a numpy
array
278         core_ids_array = np.array(list(owned_by_core_ids
))
279
280         # b) Creates a boolean mask (True if the pixel
belongs to a basin of interest)
281         mask = np.isin(basins_data, core_ids_array)
282
283         # c) Checks which noData value is used by DEM

```

```

and applies mask over basin_data
284         with rasterio.open(path_basins, 'r+') as dst:
285             lines, columns = basins_data.shape
286             area_m_square = (lines * 30) * (columns *
30)
287             print(f"        -> Total area processed: {
area_m_square / 1000000:.2f} km2")
288             nodata_val = dst.nodata if dst.nodata is not
None else 0
289
290             filtered_basins = np.where(mask, basins_data
, nodata_val)
291             dst.write(filtered_basins, 1)
292
293             # d) Open, filters and overwrites depth TIF
294             path_depth = os.path.join(out_dir,
filename_depth)
295             with rasterio.open(path_depth, 'r+') as dst:
296                 depth_data = dst.read(1)
297                 nodata_val_depth = dst.nodata if dst.nodata
is not None else 0
298
299                 filtered_depth = np.where(mask, depth_data,
nodata_val_depth)
300                 dst.write(filtered_depth, 1)
301
302             # testing quick visuals
303             print(f"        -> Generating image ...")
304             buf_bounds = (buf_minx, buf_miny, buf_maxx,
buf_maxy)
305             plot_tile_diagnostics(
306                 filtered_basins,
307                 transform,
308                 core_bounds_5800,
309                 buf_bounds,
310                 centers,
311                 unique_ids,
312                 owned_by_core_ids,
313                 tile_id,
314             )
315             sucess = True
316
317
318

```

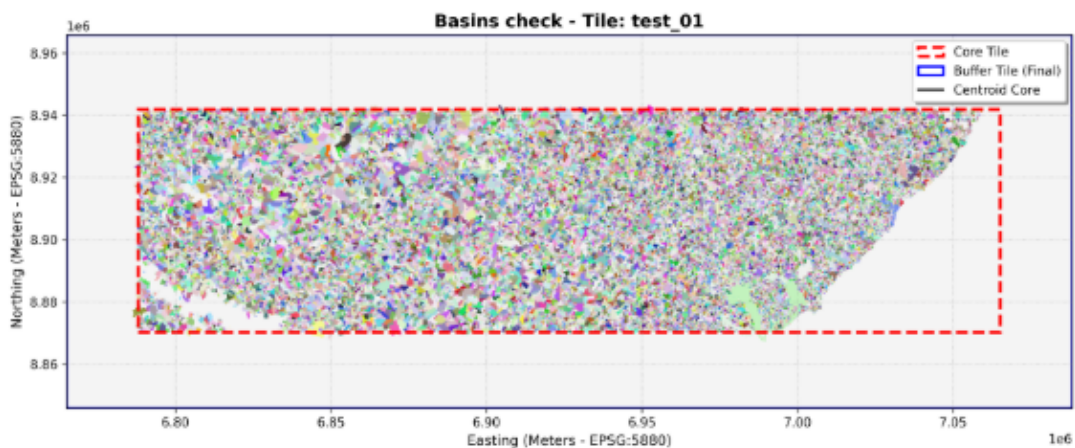
```

319         except Exception as e:
320             print(f"[ERROR]: {e}")
321             print(f"        -> Ending runtime ...")
322             break
323
324         #
325         temp_files = [filename_dem, filename_dem_clean,
326 filename_d8, filename_sinks]
327         if not sucess:
328             temp_files.extend([filename_basins, filename_depth])
329         # Deletes if failed
330
331         for f in temp_files:
332             path_f = os.path.join(out_dir, f)
333             if os.path.exists(path_f):
334                 os.remove(path_f)
335
336         if not sucess and buffer_current >= buffer_limit:
337             print(f"[WARNING]: Buffer limit reached. Consider
338 revising terrain.")
339
340 if True:
341     PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
342     OUT_DIR = 'test_basins_dynamic'
343     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
344
345     dynamic_cut(
346         path_vrt=PATH_VRT,
347         out_dir=OUT_DIR,
348         core_bounds_deg=BOUNDS_TESTING,
349         tile_id="test_01"
350     )

```

This will give us our final output for this subsection:

```
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ python3 terrain_cut_centroid.py
[test_01] Trying to process with a margin of 0.1
-> Running WhiteboxTools ...
-> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
-> Running WhiteboxTools ...
-> Running colision test ...
[SUCCESS] All interesting basins fit within a buffer of 0.2
-> Applying mask to keep only basins of interest ...
-> Total area processed: 38796.10 km2
-> Generating image ...
-> Image stored as: diagnostic_test_01.png
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$
```



As we can see, we no longer have those extra basins on our final file!

## 2.4 Analysing basins and handling infiltration

So now we have it: we can actually find basins! But how do we analyse each one of them?

Before we get anything started, we have to address the elephant in the room: rain water infiltration. As you may already guess, soil is permeable → this means that water will only start overflowing when terrain gets too saturated. Not only that, but different terrains will behave in different ways when rainwater falls, where some will soak a lot of volume but others will basically be impermeable. So this raises a question: How can we deal with this problem?

This rushes us to present another very cool thing in hydrology: the Curve Number (CN) method. It was developed by the US Department of Agriculture (USDA) Natural Resources Conservation Service (NRCS) (back when it was formerly known as the Soil Conservation Service (SCS)). The SCS CN method estimates precipitation excess as a function of the cumulative precipitation depth, soil cover, land use, and antecedent soil moisture as:

$$P_e = \begin{cases} 0, & \text{for } P < I_a \\ \frac{(P - I_a)^2}{P - I_a + S}, & \text{otherwise} \end{cases} \quad (1)$$

where  $P_e$  = accumulated precipitation excess at time  $t$ ;  $P$  = accumulated rainfall depth at time  $t$ ;  $I_a$  = the initial abstraction (initial loss); and  $S$  = potential maximum retention, a measure of the ability of a watershed to abstract and retain storm precipitation. Until the accumulated rainfall exceeds the initial abstraction, the precipitation excess, hence the runoff, will be zero. From analysis of results from many small experimental watersheds, the SCS developed an empirical relation of  $I_a$  and  $S$ :

$$I_a = 0.2 * S \quad (2)$$

Therefore, the cumulative excess as time  $t$  is:

$$P_e = \frac{(P - 0.2 * S)^2}{(P + 0.8 * S)} \quad (3)$$

Incremental excess for a time interval is computed as the difference between the accumulated excess at the end of and beginning of the period. The maximum retention,  $S$ , and watershed characteristics are related through an intermediate parameter, the curve number (commonly abbreviated CN) as:

$$S = \begin{cases} \frac{1000 - 10 * CN}{CN}, & \text{foot / pound system} \\ \frac{25400 - 254 * CN}{CN}, & \text{The obviously better alternative, also known as SI} \end{cases} \quad (4)$$

Jokes aside, CN values range from 100 (for water bodies) to approximately 30 for permeable soils with high infiltration rates.

The curve number that is entered should be a "composite" curve number that represents all of the different soil groups and land use combinations in the subbasin. Typically, curve numbers are derived from soil maps.  $I_a$  defines the amount of precipitation that must fall before excess precipitation results. Here, we will use (2) (as provided by the curve number provider that we will mention shortly).

For a watershed that consists of several soil types and land uses, a composite CN is calculated as:

$$CN_{composite} = \frac{\sum A_i CN_i}{\sum A_i} \quad (5)$$

In which  $CN_{composite}$  = the composite CN used for runoff volume computations;  $i$  = and index of watersheds subdivisions of uniform land use and soil type;  $CN_i$  = the CN subdivisions  $i$ ; and  $A_i$  = the drainage area of subdivision  $i$ .

We need to have this ideas in mind, but we can leave some of them aside for now, since we'll now figure out how to analyse each individual basin. First, we have to look at our last program's outputs (in this case, the tif files):

- basins final  $\rightarrow$  tells us where basins are.
- depth final  $\rightarrow$  tells us the max depth of each of the basins pieces until it overflows.

Here, the secret is to use NumPy masks. We tell Python to isolate only the pixels belonging to a specific ID, and we run calculations with only those pixels. Then, we will convert the CN of every pixel from AMC (antecedent moisture condition) II, equal to normal moisture conditions, to AMC III (saturated soil). We'll do this so that the worst case scenario of a flood caused by a storm that came when the soil was already saturated from previous rain (that is a completely plausible and possible scenario) also gets considered in our calculations.

After all of that is done, we will use the adjusted CN values to calculate the surface runoff (the amount of water that doesn't soak into the ground). Because NumPy works with arrays, we can apply the SCS-CN formulas to all of our different storm scenarios at the same time. By multiplying this runoff by the basin's drainage area, we get the total volume of water. and storing all these results into a CSV table for easy reading.

Another interesting point to mention is how we calculate our composite CN. Let's return to Equation (5):

$$CN_{composite} = \frac{\sum A_i CN_i}{\sum A_i} \quad (6)$$

However, we have a key detail to notice: our CN data is stored in a raster file (.tif) where space is represented by a bunch of pixels.

Taking  $n$  as the number of distinct soil/land-use categories,  $A_{pixel}$  as the individual area of a single pixel,  $p_i$  as the number of pixels belonging to category  $i$ , and  $N_{total}$  as the total number of pixels in the watershed ( $\sum_{i=1}^n p_i = N_{total}$ ), we can rewrite the expression as:

$$\begin{aligned}
CN_{composite} &= \frac{\sum A_i CN_i}{\sum A_i} \\
&= \frac{A_{pixel} \cdot p_1 CN_1 + \dots + A_{pixel} \cdot p_n CN_n}{A_{pixel} \cdot N_{total}} \\
&= \frac{A_{pixel} \cdot (p_1 CN_1 + p_2 CN_2 + \dots + p_n CN_n)}{A_{pixel} \cdot N_{total}} \\
&= \frac{\sum_{i=1}^n p_i CN_i}{N_{total}} \\
&= \text{arithmetic mean of all individual pixel CN values}
\end{aligned} \tag{7}$$

So, with all of that in mind, let's write a new function to our previous code:

```

1 def simulate_basin_volumes_worst_case(path_basins, path_depth,
2   path_cn, owned_ids, out_dir, tile_id):
3     """
4     Analyses individual basins, and for each one calculates:
5     -> Composite CN (after we convert each one from AMC II to
6     AMC III)
7     -> Volume / Water height ratio
8     -> Other important stuff
9
10    and then stores the values in a cute table!
11
12    :param path_basins: Path to our basins_final .tif file
13    :param path_depth: Path to our depth_final .tif file
14    :param path_cn: Path to our CN table .tif file (observation:
15    their CN values are AMC II standard. We must convert them
16    first)
17    :param owned_ids: Our owned_by_core_ids variable (go to the
18    main function if in doubt)
19    :param out_dir: Our output directory
20    :param tile_id: Our tile id (go to the main function if in
21    doubt)
22
23    """
24
25    # saves different storm strenghts to test (in mm)
26    # according to opticalscientific.com:
27    # light rain: < 60mm / 24hrs ( < 2.5mm / hour, and we
28    converted it to 24hrs)
29    # moderate rain: 60 to 180mm / 24hrs (2.5 to 7.5 mm / hour)
30    # heavy rain: 180mm to 1200mm / 24hrs (7.5 to 50mm / hour)
31    # Violent rain: > 1200 / 24hrs ( > 50mm / hour)

```



```

25     # here, we will add some in-between values for calculations'
      sake
26     scenarios = np.array([60, 180, 300, 600, 800, 1200], dtype=
float)
27
28     all_dfs = []
29
30     with rasterio.open(path_basins) as src_b, \
31         rasterio.open(path_depth) as src_d, \
32         rasterio.open(path_cn) as src_cn:
33
34         # Validates dimentions (between our CN and DEM)
35         if not (src_b.shape == src_d.shape == src_cn.shape):
36             raise ValueError(
37                 f"[ALIGNMENT ERROR] Dimentions are not
compatible\n"
38                 f"         -> Basins: {src_b.shape}\n"
39                 f"         -> Depth:  {src_d.shape}\n"
40                 f"         -> CN:      {src_cn.shape}"
41             )
42
43         # 2. Validates resolution and geographical positioning (
Geotransform)
44         # Allows a tiny tiny tolerance for floating point Warp
imperfections
45         if not np.allclose(src_b.transform, src_cn.transform,
atol=1e-5):
46             raise ValueError(
47                 "[ALIGNMENT ERROR] The matrixes cover entirely
different areas or have different spacial resolution"
48             )
49
50         basins_data = src_b.read(1)
51         depth_data = src_d.read(1)
52         cn_data = src_cn.read(1)
53
54         pixel_width = src_b.transform[0]
55         pixel_height = -src_b.transform[4]
56         pixel_area = pixel_width * pixel_height
57
58         for basin_id in owned_ids:
59             mask = (basins_data == basin_id)
60             if not np.any(mask):
61                 continue

```

```

62         basin_area = np.sum(mask) * pixel_area
63
64         valid_cn = cn_data[mask]
65         valid_cn = valid_cn[valid_cn > 0]
66         # Converts array to float to prevent uint8 overflow
67
68         if len(valid_cn) == 0:
69             print(f"        -> Skipping basin {basin_id}: No
70 valid CN data.")
71             continue
72
73         valid_cn_float = valid_cn.astype(float)
74
75         # converts entire matrix from AMC II to AMC III
76 pixel by pixel
77         valid_cn_iii = (23 * valid_cn_float) / (10 + 0.13 *
78 valid_cn_float)
79
80         # calculates composite already on the worst case
81 scenario
82         cn_iii = np.mean(valid_cn_iii)
83         cn_ii_mean_original = np.mean(valid_cn)
84
85         # SCS CN with saturated soil
86         S = (25400 / cn_iii) - 254
87         Q = np.zeros_like(scenarios)
88
89         valid_p = scenarios > (0.2 * S)
90         Q[valid_p] = ((scenarios[valid_p] - 0.2 * S)**2) / (
91 scenarios[valid_p] + 0.8 * S)
92
93         volume_rain_m3 = (Q / 1000) * basin_area
94
95         depths = depth_data[mask]
96         max_depth = depths.max()
97
98         h_range = np.linspace(0, max_depth, 50)
99         v_range = []
100
101         for h in h_range:
102             water_depth = np.maximum(0, h - (max_depth -
103 depths))
104             v_range.append(np.sum(water_depth) * pixel_area)

```

```

100         final_heights = np.interp(volume_rain_m3, v_range,
101         h_range)
102
103         df = pd.DataFrame({
104             'Basin_ID': int(basin_id),
105             'Precipitation_mm': scenarios,
106             'AMC_II_Original': np.round(cn_ii_mean_original,
107         2),
108             'AMC_III_Saturated': np.round(cn_iii, 2),
109             'Runoff_Q_mm': np.round(Q, 2),
110             'Volume_m3': np.round(volume_rain_m3, 2),
111             'Max_height_m': np.round(final_heights, 2)
112         })
113
114         all_dfs.append(df)
115
116     if all_dfs:
117         combined_df = pd.concat(all_dfs, ignore_index=True)
118         csv_path = os.path.join(out_dir, f'all_basins_{tile_id}
119         _wcs.csv')
120         combined_df.to_csv(csv_path, index=False)
121         print(f"    -> Exported worst-case volume table:
122         all_basins_{tile_id}_wcs.csv")

```

But before continuing to implement this new function on the final code, we also have to deal with another thing: How will we know where the basin is in the real world if we only store volume related values and CN numbers? For that, we will also need to modify our cut function from the previous subsection. For keeping things simple, we'll just store our spacial info by storing it as a GeoPackage, a vectorial and usually lightweight file format. With both our CSV table and our GeoPackage sharing a basin id column, we can later access the volume-related data on the space we need to work on.

With all of that in mind, let's convert that to code:

```

1 import os
2 import geopandas as gpd
3 from shapely.geometry import box, shape
4 from osgeo import gdal

```

```

5 from whitebox import WhiteboxTools
6 import rasterio
7 import rasterio.features
8 import numpy as np
9 import pandas as pd
10 import scipy.ndimage as ndimage
11 import matplotlib.pyplot as plt
12 import matplotlib.patches as patches
13 from matplotlib.colors import ListedColormap
14
15 gdal.UseExceptions()
16
17 def simulate_basin_volumes_worst_case(path_basins, path_depth,
18     path_cn, owned_ids, out_dir, tile_id):
19     """
20     Analyses individual basins, and for each one calculates:
21     -> Composite CN (after we convert each one from AMC II to
22     AMC III)
23     -> Volume / Water height ratio
24     -> Other important stuff
25
26     and then stores the values in a cute table!
27
28     :param path_basins: Path to our basins_final .tif file
29     :param path_depth: Path to our depth_final .tif file
30     :param path_cn: Path to our CN table .tif file (observation:
31     their CN values are AMC II standard. We must convert them
32     first)
33     :param owned_ids: Our owned_by_core_ids variable (go to the
34     main function if in doubt)
35     :param out_dir: Our output directory
36     :param tile_id: Our tile id (go to the main function if in
37     doubt)
38
39     """
40
41     # saves different storm strenghts to test (in mm)
42     # according to opticalscientific.com:
43     # light rain: < 60mm / 24hrs ( < 2.5mm / hour, and we
44     converted it to 24hrs)
45     # moderate rain: 60 to 180mm / 24hrs (2.5 to 7.5 mm / hour)
46     # heavy rain: 180mm to 1200mm / 24hrs (7.5 to 50mm / hour)
47     # Violent rain: > 1200 / 24hrs ( > 50mm / hour)
48     # here, we will add some in-between values for calculations'

```

```

sake
42     scenarios = np.array([60, 180, 300, 600, 800, 1200], dtype=
float)
43
44     all_dfs = []
45
46     with rasterio.open(path_basins) as src_b, \
47         rasterio.open(path_depth) as src_d, \
48         rasterio.open(path_cn) as src_cn:
49
50         # Validates dimentions (between our CN and DEM)
51         if not (src_b.shape == src_d.shape == src_cn.shape):
52             raise ValueError(
53                 f"[ALIGNMENT ERROR] Dimentions are not
compatible\n"
54                 f"         -> Basins: {src_b.shape}\n"
55                 f"         -> Depth:  {src_d.shape}\n"
56                 f"         -> CN:      {src_cn.shape}"
57             )
58
59         # 2. Validates resolution and geographical positioning (
Geotransform)
60         # Allows a tiny tiny tolerance for floating point Warp
imperfections
61         if not np.allclose(src_b.transform, src_cn.transform,
atol=1e-5):
62             raise ValueError(
63                 "[ALIGNMENT ERROR] The matrixes cover entirely
different areas or have different spacial resolution"
64             )
65
66         basins_data = src_b.read(1)
67         depth_data = src_d.read(1)
68         cn_data = src_cn.read(1)
69
70         pixel_width = src_b.transform[0]
71         pixel_height = -src_b.transform[4]
72         pixel_area = pixel_width * pixel_height
73
74         for basin_id in owned_ids:
75             mask = (basins_data == basin_id)
76             if not np.any(mask):
77                 continue
78

```

```

79         basin_area = np.sum(mask) * pixel_area
80
81         valid_cn = cn_data[mask]
82         valid_cn = valid_cn[valid_cn > 0]
83         # Converts array to float to prevent uint8 overflow
84
85         if len(valid_cn) == 0:
86             print(f"        -> Skipping basin {basin_id}: No
valid CN data.")
87             continue
88
89         valid_cn_float = valid_cn.astype(float)
90
91         # converts entire matrix from AMC II to AMC III
pixel by pixel
92         valid_cn_iii = (23 * valid_cn_float) / (10 + 0.13 *
valid_cn_float)
93
94         # calculates composite already on the worst case
scenario
95         cn_iii = np.mean(valid_cn_iii)
96         cn_ii_mean_original = np.mean(valid_cn)
97
98         # SCS CN with saturated soil
99         S = (25400 / cn_iii) - 254
100         Q = np.zeros_like(scenarios)
101
102         valid_p = scenarios > (0.2 * S)
103         Q[valid_p] = ((scenarios[valid_p] - 0.2 * S)**2) / (
scenarios[valid_p] + 0.8 * S)
104
105         volume_rain_m3 = (Q / 1000) * basin_area
106
107         depths = depth_data[mask]
108         max_depth = depths.max()
109
110         h_range = np.linspace(0, max_depth, 50)
111         v_range = []
112
113         for h in h_range:
114             water_depth = np.maximum(0, h - (max_depth -
depths))
115             v_range.append(np.sum(water_depth) * pixel_area)
116

```

```

117         final_heights = np.interp(volume_rain_m3, v_range,
118                                     h_range)
119
120         df = pd.DataFrame({
121             'Basin_ID': int(basin_id),
122             'Precipitation_mm': scenarios,
123             'AMC_II_Original': np.round(cn_ii_mean_original,
124                                         2),
125             'AMC_III_Saturated': np.round(cn_iii, 2),
126             'Runoff_Q_mm': np.round(Q, 2),
127             'Volume_m3': np.round(volume_rain_m3, 2),
128             'Max_height_m': np.round(final_heights, 2)
129         })
130
131         all_dfs.append(df)
132
133     if all_dfs:
134         combined_df = pd.concat(all_dfs, ignore_index=True)
135         csv_path = os.path.join(out_dir, f'all_basins_{tile_id}_wcs.csv')
136         combined_df.to_csv(csv_path, index=False)
137         print(f"    -> Exported worst-case volume table:
138               all_basins_{tile_id}_wcs.csv")
139
140 def plot_tile_diagnostics(
141     basins_data,
142     transform,
143     core_bounds_5800,
144     buf_deg_bounds,
145     centers,
146     unique_ids,
147     owned_by_core_ids,
148     tile_id,
149 ):
150     """
151     Takes our results from the dynamic cutting and outputs them
152     as a visual .png file so we can have visual
153     proof that all centroids are contained inside the core box
154     It is advised to read the dynamic_cut function before
155     reading this one (down below).
156     """
157
158     # WARNING: DO NOT REMOVE FIG.

```

```

155     fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
156
157     # 1. Generating unique colors for each ID
158     # Generates random colors for each ID to try to avoid visual
159     # problems with neighbor basins
160     max_id = int(basins_data.max())
161     np.random.seed(42)
162     color_array = np.random.rand(max_id + 1, 4)
163     color_array[0] = [0, 0, 0, 0] # Defines background color (
164     ID 0 = noData)
165     custom_cmap = ListedColormap(color_array)
166
167     # 2. Plots basins raster with custom colors
168     im = ax.imshow(
169         basins_data,
170         cmap=custom_cmap,
171         interpolation='nearest', # Avoids blurring
172         extent=[
173             transform[2],
174             transform[2] + transform[0] * basins_data.shape[1],
175             transform[5] + transform[4] * basins_data.shape[0],
176             transform[5],
177         ],
178     )
179
180     buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
181     df_buf = gpd.GeoDataFrame(
182         {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy)
183         ]}],
184         crs='EPSG:4326',
185     )
186
187     b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
188     total_bounds
189
190     c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
191
192     # Draws Core Box limits
193     core_rect = patches.Rectangle(
194         (c_minx, c_miny),
195         c_maxx - c_minx,
196         c_maxy - c_miny,
197         linewidth=2.5,
198         edgecolor='red',
199         facecolor='none',
200         linestyle='--',

```



```

195         label='Core Tile',
196     )
197     ax.add_patch(core_rect)
198
199     # Draws Core Box + Buffer tile
200     buf_rect = patches.Rectangle(
201         (b_minx, b_miny),
202         b_maxx - b_minx,
203         b_maxy - b_miny,
204         linewidth=2,
205         edgecolor='blue',
206         facecolor='none',
207         label='Buffer Tile (Final)',
208     )
209     ax.add_patch(buf_rect)
210
211     # Plots centroids
212     first_core = True
213     for basin_id, (row_center, col_center) in zip(unique_ids,
214 centers):
215         x_coord, y_coord = transform * (col_center, row_center)
216
217         if basin_id in owned_by_core_ids:
218             label_text = 'Centroid Core' if first_core else ''
219             ax.plot(
220                 x_coord,
221                 y_coord,
222                 marker='o', # <-- Alterado para o ponto preto
223                 color='black',
224                 markersize=5,
225                 markeredgecolor='white',
226                 markeredgewidth=0.5,
227                 label=label_text,
228             )
229             first_core = False
230
231         else:
232             ax.plot(
233                 x_coord,
234                 y_coord,
235                 marker='o', # <-- Alterado para o ponto cinza
236                 color='dimgray',

```

```

236         markersize=2,
237         alpha=0.4,
238     )
239
240     # Adjust zoom
241     ax.set_xlim(b_minx, b_maxx)
242     ax.set_ylim(b_miny, b_maxy)
243
244     # Graphic aesthetics
245     ax.set_facecolor('#f4f4f4')
246     ax.grid(True, linestyle=':', alpha=0.5, color='gray')
247     plt.title(
248         f'Basins check - Tile: {tile_id}',
249         fontsize=14,
250         fontweight='bold',
251     )
252     plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
253     plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
254     plt.legend(loc='upper right', frameon=True, facecolor='white', shadow=True)
255
256     plt.tight_layout()
257     plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
258     plt.close()
259     print(
260         f'    -> Image stored as: diagnostic_{tile_id}.png'
261     )
262
263 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
264     """
265     This will cut the DEM according to topography, preserving
266     basin boundaries (AKA no basins get cut)
267     Criteria for cutting: All basins with a centroid that
268     belongs to core tile A will be grouped together on the same
269     TIF
270     :param path_vrt: Path to our .vrt file
271     :param out_dir: Path for our output
272     :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
273     :param tile_id: Unique tile ID
274
275     """
276
277     os.makedirs(out_dir, exist_ok=True)
278     minx, miny, maxx, maxy = core_bounds_deg

```

```

276
277     # Projects Core Tile limist from 4326 (degress) to 5800 (
278     # meters)
279     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
280     maxx, maxy)]}, crs="EPSG:4326")
281     df_core_5800 = df_core.to_crs("EPSG:5880")
282     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
283     maxx, maxy) em metros
284
285     # Dynamic buffer initial setup
286     buffer_current = 0.1
287     buffer_steps = 0.1
288     buffer_limit = 4
289     sucess = False
290
291     wbt = WhiteboxTools()
292     wbt.set_working_dir(os.path.abspath(out_dir))
293     wbt.verbose = False
294
295     while not sucess and buffer_current <= buffer_limit:
296         print(f"[{tile_id}] Trying to process with a margin of {
297         buffer_current}")
298
299         # Define limits
300         buf_minx = minx - buffer_current
301         buf_miny = miny - buffer_current
302         buf_maxx = maxx + buffer_current
303         buf_maxy = maxy + buffer_current
304
305         # Names of temporary files
306         filename_dem = f"temp_{tile_id}_dem.tif"
307         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
308         filename_d8 = f"temp_{tile_id}_d8.tif"
309         filename_sinks = f"temp_{tile_id}_sinks.tif"
310         filename_basins = f"basins_final_{tile_id}.tif"
311         filename_depth = f"depth_final_{tile_id}.tif"
312         filename_cn = f"temp_{tile_id}_cn.tif"
313
314         path_dem = os.path.join(out_dir, filename_dem)
315         path_basins = os.path.join(out_dir, filename_basins)
316         path_cn_temp = os.path.join(out_dir, filename_cn)
317
318         try:
319             # Cuts OG VRT

```

```

316         gdal.Warp(
317             destNameOrDestDS=path_dem,
318             srcDSOrSrcDSTab=path_vrt,
319             format='GTiff',
320             outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
321             outputBoundsSRS='EPSG:4326',
322             dstSRS='EPSG:5880',
323             xRes=30, yRes=30,
324             creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]
325         )
326
327     # Cuts our CN
328     path_cn_original = "BR_CN-2022.tif"
329     gdal.Warp(
330         destNameOrDestDS=path_cn_temp,
331         srcDSOrSrcDSTab=path_cn_original,
332         format='GTiff',
333         outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
334         outputBoundsSRS='EPSG:4326',
335         dstSRS='EPSG:5880',
336         xRes=30, yRes=30,
337         dstNodata=0, # <-- makes sure to have 0's on
noDATA zones
338         creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]
339     )
340
341     # WhiteboxTools pipeline here
342     print(f"        -> Running WhiteboxTools ...")
343     wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)
344     wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
345     wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
346     wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
347
348     # Collision Test pipeline here
349
350     print(f"        -> Running collision test ...")

```

```

351         with rasterio.open(path_basins) as src:
352             basins_data = src.read(1)
353             transform = src.transform
354
355         # a) Grabs IDS from all basins that touch the borders
356         of the current matrix (core box + current buffer)
357         top_edge = basins_data[0, :]
358         bottom_edge = basins_data[-1, :]
359         left_edge = basins_data[:, 0]
360         right_edge = basins_data[:, -1]
361
362         edge_ids = set(np.unique(np.concatenate([top_edge,
363         bottom_edge, left_edge, right_edge])))
364         edge_ids.discard(0) # Removes noData
365
366         # b) Calculates the centroid of each basin present
367         inside the raster file
368         unique_ids = np.unique(basins_data)
369         unique_ids = unique_ids[unique_ids > 0] # Removes
370         noData (I hope)
371
372         centers = ndimage.center_of_mass(basins_data > 0,
373         basins_data, unique_ids)
374
375         owned_by_core_ids = set()
376         c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
377
378         # c) Filters which basins have their centroid INSIDE
379         core tile
380         for basin_id, (row_center, col_center) in zip(
381         unique_ids, centers):
382             # Converts pixel (line, column) to geo cords (x,
383             y in meters)
384             x_coord, y_coord = transform * (col_center,
385             row_center)
386
387             if (c_minx <= x_coord <= c_maxx) and (c_miny <=
388             y_coord <= c_maxy):
389                 owned_by_core_ids.add(basin_id)
390
391         # d) Is there any basin with centroid inside core
392         tile that leaks outside of the buffer?
393         leaking_basins = owned_by_core_ids.intersection(

```

```

edge_ids)
384
385         if len(leaking_basins) > 0:
386             print(f"[FAIL] {len(leaking_basins)} basin(s)
leaked outside current buffer")
387             buffer_current += buffer_steps
388
389         else:
390             print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
391
392             wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
393
394             # Removing excess basins
395             print(f"        -> Applying mask to keep only basins
of interest ...")
396
397             # a) Transforms the valid IDs set into a numpy
array
398             core_ids_array = np.array(list(owned_by_core_ids
))
399
400             # b) Creates a boolean mask (True if the pixel
belongs to a basin of interest)
401             mask = np.isin(basins_data, core_ids_array)
402
403             # c) Checks which noData value is used by DEM
and applies mask over basin_data
404             with rasterio.open(path_basins, 'r+') as dst:
405                 lines, columns = basins_data.shape
406                 area_m_square = (lines * 30) * (columns *
30)
407                 print(f"        -> Total area processed: {
area_m_square / 1000000:.2f} km2")
408                 nodata_val = dst.nodata if dst.nodata is not
None else 0
409
410                 filtered_basins = np.where(mask, basins_data
, nodata_val)
411                 dst.write(filtered_basins, 1)
412
413             # d) Open, filers and overwrites depth TIF
414             path_depth = os.path.join(out_dir,

```

```

filename_depth)
415         with rasterio.open(path_depth, 'r+') as dst:
416             depth_data = dst.read(1)
417             nodata_val_depth = dst.nodata if dst.nodata
is not None else 0
418
419             filtered_depth = np.where(mask, depth_data,
nodata_val_depth)
420             dst.write(filtered_depth, 1)
421
422             # testing quick visuals
423             print(f"        -> Generating image ...")
424             buf_bounds = (buf_minx, buf_miny, buf_maxx,
buf_maxy)
425             plot_tile_diagnostics(
426                 filtered_basins,
427                 transform,
428                 core_bounds_5800,
429                 buf_bounds,
430                 centers,
431                 unique_ids,
432                 owned_by_core_ids,
433                 tile_id,
434             )
435
436             # calculating basin geometry and volume / water
height table, alongside some other
437             # important stuff
438
439             print(f"        -> Calculating worst-case
hydrological scenarios\n    Alonside some other important stuff
...")
440
441             shapes_gen = rasterio.features.shapes(
442                 filtered_basins.astype(np.int32),
443                 transform=transform
444             )
445
446             records = []
447
448             for geom, value in shapes_gen:
449                 if value > 0 and int (value) in
owned_by_core_ids:
450                     poly = shape(geom)

```

```

451         records.append({
452             'basin_id': int(value),
453             'geometry': poly
454         })
455
456     if records:
457         gdf_basins = gpd.GeoDataFrame(records, crs="
EPSG:5880")
458         gdf_basins = gdf_basins.dissolve(by='
basin_id', as_index=False)
459         gdf_basins['area_m2'] = gdf_basins.geometry.
area
460
461         gpkg_path = os.path.join(out_dir, f'
core_basins_{tile_id}.gpkg')
462         gdf_basins.to_file(gpkg_path, driver="GPKG")
463         print(f"        -> Core basin vector stored as:
core_basins_{tile_id}.gpkg")
464
465         simulate_basin_volumes_worst_case(
466             path_basins=path_basins,
467             path_depth=path_depth,
468             path_cn=path_cn_temp,
469             owned_ids=owned_by_core_ids,
470             out_dir=out_dir,
471             tile_id=tile_id
472         )
473         sucess = True
474
475     except Exception as e:
476         print(f"[ERROR]: {e}")
477         print(f"        -> Ending runtime ...")
478         break
479
480     # Limpeza incluindo o filename_cn!
481     temp_files = [filename_dem, filename_dem_clean,
filename_d8, filename_sinks, filename_cn]
482     if not sucess:
483         temp_files.extend([filename_basins, filename_depth])
484     # Deletes if failed
485
486     for f in temp_files:
487         path_f = os.path.join(out_dir, f)
488         if os.path.exists(path_f):

```



```

488         os.remove(path_f)
489
490     if not sucess and buffer_current >= buffer_limit:
491         print(f"[WARNING]: Buffer limit reached. Consider
revising terrain.")
492
493 if __name__ == "__main__":
494     PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
495     OUT_DIR = 'test_basins_dynamic'
496     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
497
498     dynamic_cut(
499         path_vrt=PATH_VRT,
500         out_dir=OUT_DIR,
501         core_bounds_deg=BOUNDS_TESTING,
502         tile_id="test_01"
503     )
504

```

For this new code, we get the following results:

```

[test_01] Trying to process with a margin of 0.1
-> Running WhiteboxTools ...
-> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
-> Running WhiteboxTools ...
-> Running colision test ...
[SUCCESS] All interesting basins fit within a buffer of 0.2
-> Applying mask to keep only basins of interest ...
-> Total area processed: 38796.10 km2
-> Generating image ...
-> Image stored as: diagnostic_test_01.png
-> Calculating worst-case hydrological scenarios
Alongside some other important stuff ...
-> Core basin vector stored as: core_basins_test_01.gpkg
-> Skipping basin 3: No valid CN data.
-> Exported worst-case volume table: all_basins_test_01_wcs.csv

```

```

✓ test_basins_dynamic
  all_basins_test_01_wcs.csv
  basins_final_test_01.tif
  core_basins_test_01.gpkg
  depth_final_test_01.tif

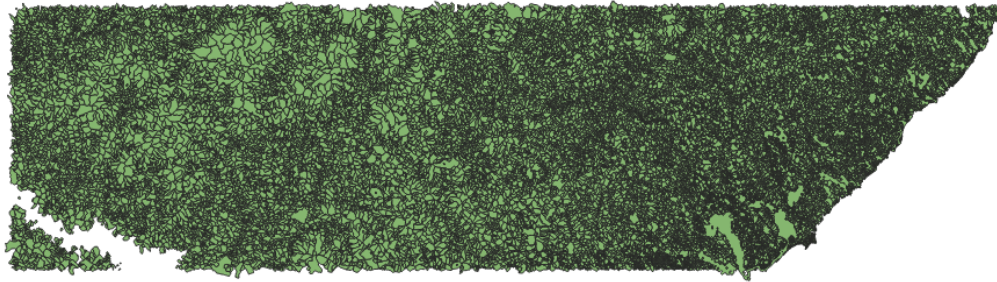
```

```

Basin_ID,Precipitation_mm,AMC_II_Original,AMC_III_Saturated,Runoff_Q_mm,Volume_m3,Max_height_m
2,60.0,89.0,94.9,46.25,5286.13,1.75
2,180.0,89.0,94.9,164.6,18813.41,2.74
2,300.0,89.0,94.9,284.22,32486.36,3.36
2,600.0,89.0,94.9,583.93,66742.73,4.47
2,800.0,89.0,94.9,783.85,89594.13,5.06
2,1200.0,89.0,94.9,1183.77,135305.46,5.38
4,60.0,89.0,94.9,46.25,1831.42,0.96
4,180.0,89.0,94.9,164.6,6518.03,1.79
4,300.0,89.0,94.9,284.22,11255.12,2.15
4,600.0,89.0,94.9,583.93,23123.46,2.35
4,800.0,89.0,94.9,783.85,31040.49,2.35
4,1200.0,89.0,94.9,1183.77,46877.48,2.35
5,60.0,88.65,93.71,43.48,5574820.83,1.44
5,180.0,88.65,93.71,161.03,20646553.58,1.62
5,300.0,88.65,93.71,280.46,35958515.92,1.79
5,600.0,88.65,93.71,580.0,74364558.65,1.9
5,800.0,88.65,93.71,779.89,99992412.89,1.9
5,1200.0,88.65,93.71,1179.77,151262899.07,1.9
6,60.0,89.0,94.9,46.25,10031.16,4.32
6,180.0,89.0,94.9,164.6,35701.03,6.37
6,300.0,89.0,94.9,284.22,61647.34,7.19
6,600.0,89.0,94.9,583.93,126653.52,7.66
6,800.0,89.0,94.9,783.85,170017.21,7.66
6,1200.0,89.0,94.9,1183.77,256760.75,7.66
8,60.0,89.0,94.9,46.25,13902.11,1.54
8,180.0,89.0,94.9,164.6,49477.77,2.61
8,300.0,89.0,94.9,284.22,85436.56,3.37
8,600.0,89.0,94.9,583.93,175528.11,4.61
8,800.0,89.0,94.9,783.85,235625.51,4.82
8,1200.0,89.0,94.9,1183.77,355842.69,4.82
9,60.0,69.65,82.79,23.91,4260.1,0.74

```

And for our GeoPackage:



So there you have it! We can:

- Find Bowl-like depressions
- Calculate volume relationships
- Maintain a map of where basins are so we can use that data later with  $O(1)$  complexity

But there is still something we are leaving out: We are so far only treating a specific core box that has been chosen by us. We now have to tweak our code a little so that we can start handling the entire DEM. But that is already a new subsection!

## 2.5 Applying results for the entire DEM